Абстрактні класи й інтерфейси

№ уроку:	4	**Курс:**	Java Essential

Засоби навчання:	Комп'ютер із встановленою IntelliJ IDEA

Огляд, мета та призначення уроку

- Розгляд абстрактних класів.
- Розгляд інтерфейсів.

Вивчивши матеріал цього заняття, учень зможе:

- Використовувати та створювати абстрактні класи.
- Використовувати та створювати інтерфейси.

Зміст уроку

1. Абстрактні класи.
2. Інтерфейси.
3. Паттерн впровадження залежності.

Резюме

- **Абстракція** в об'єктно-орієнтованому програмуванні – це надання об'єктові характеристик, які відрізняють його від інших об'єктів, чітко визначаючи його концептуальні межі. Основна ідея полягає в тому, щоб відокремити спосіб використання складових об'єктів даних від деталей їхньої реалізації у вигляді простіших об'єктів подібно до того, як функціональна абстракція поділяє спосіб використання функції та деталей її реалізації у термінах примітивніших функцій. У такий спосіб дані обробляються функцією високого рівня за допомогою виклику функцій низького рівня.
- **Абстрактний клас** в об'єктно-орієнтованому програмуванні – базовий клас, який не передбачає створення екземплярів. Абстрактні класи на практиці реалізують один із принципів ООП – поліморфізм. Абстрактний клас може містити (і не містити) абстрактні методи та властивості. Абстрактний метод не реалізується для класу, в якому описаний, проте має бути реалізований для його неабстрактних нащадків. Абстрактні класи – найбільш загальні абстракції, тобто мають найбільший обсяг і найменший зміст.
- Ключове слово abstract може використовуватися з **класами та методами**.
- Ключове слово abstract під час створення класу вказує, що клас призначений лише для використання як базовий клас для інших класів. Члени, які позначені як абстрактні або додані до абстрактного класу, мають бути реалізовані за допомогою класів, похідних від абстрактних класів.
- **Можливості абстрактних класів:**

1. Не можна створити екземпляр абстрактного класу.
2. Абстрактні класи можуть містити як абстрактні, так і прості (неабстрактні) члени.
3. Неабстрактний клас, що є похідним від абстрактного, має містити фактичні реалізації всіх наслідуваних абстрактних методів і методів доступу.

- **Можливості абстрактних методів:**

1. Абстрактний метод є неявним методом.
2. Створення абстрактних методів допускається лише в абстрактних класах.
3. Тіло абстрактного методу відсутнє; створення методу просто закінчується крапкою з комою, після сигнатури ставити фігурні дужки {} не потрібно.

CyberBionic systematics
CyberBionic Systematics ®
19 Eugene Sverstyuk Str., 5 floor
Kyiv, Ukraine
t. 0 800 337 146
E-mail: edu@cbsystematics.com
Site: www.edu.cbsystematics.com | itvdn.com

4. Реалізація надається методом перевизначення override, що є членом неабстрактного класу.

- Абстрактний клас має надавати реалізацію для всіх членів інтерфейсу.
- Абстрактний клас, який реалізує інтерфейс, може зображати методи інтерфейсу в абстрактних методах.
- **Переваги використання абстрактних класів:**

1. Загальний код в одній реалізації у вигляді конкретних та абстрактних членів.
2. Зміна значення полів або неабстрактних членів абстрактного класу призводить до відповідної зміни у всіх його похідних класах.
3. Наявність реалізації за замовчуванням.

- **Інтерфейс** (від лат. inter – «між», і face – «поверхня») – семантична та синтаксична конструкція в коді програми, що використовується для специфікування послуг, які надаються класом або компонентом. Інтерфейс визначає межу взаємодії між класами або компонентами, специфікуючи певну абстракцію, яку здійснює реалізувальна сторона. На відміну від багатьох інших різновидів інтерфейсів, інтерфейс ООП є суворо формалізованим елементом об'єктно-орієнтованої мови і як семантична конструкція широко використовується кодом програми.
- Неможливо створити екземпляр інтерфейсу.
- Інтерфейси та члени інтерфейсів є абстрактними. Інтерфейси не мали реалізації за замовчуванням до версії Java 8.
- Інтерфейс містив лише сигнатури **методів** до версії Java 8.
- **Члени інтерфейсів** автоматично є **відкритими, абстрактними**. Вони **не можуть мати модифікатори доступу**.
- Поля інтерфейсів автоматично є **public static final**.
- Клас, який реалізує інтерфейс, має реалізувати члени цього інтерфейсу, які вказані під час його створення.
- Однак якщо базовий клас реалізує інтерфейс, похідний клас також успадковує цю реалізацію.
- Інтерфейс може успадковуватись від одного або декількох базових інтерфейсів.
- Базовий клас може реалізувати члени інтерфейсу. У цьому випадку похідний клас може змінити поведінку інтерфейсу через перевизначення цих членів.
- Якщо клас реалізує два інтерфейси, що містять член з однаковою сигнатурою, то під час реалізації цього члена в класі обидва інтерфейси будуть використовувати цей член для своєї реалізації.
- Переваги використання інтерфейсів:

1. Клас може реалізувати кілька інтерфейсів.
2. Якщо клас реалізує інтерфейс, він отримує лише імена та сигнатури методу.
3. Інтерфейси визначають поведінку екземплярів похідних класів.
4. Базовий клас може мати непотрібний функціонал, отриманий від інших його базових класів. Цього можна уникнути, застосовуючи інтерфейси.

- **Впровадження залежностей** (Dependency Injection, DI) – чудова методика для створення слабозв'язаних застосунків. Вона надає можливості для спрощення коду, вилучення та обробки залежностей між об'єктами й автоматичного створення екземплярів залежного об'єкта.
- Впровадження залежностей описує процес розробки застосунків – замість зазначення конкретних залежностей у застосунку під час розробки та створення необхідних об'єктів у коді під час виконання застосунок вирішує, які об'єкти йому потрібні, а потім створює та впроваджує їх у застосунок.
- Використання впровадження залежностей має кілька переваг:

1. Ослаблення зв'язку між класами.
2. Створення коду, який краще піддається перевірці.
3. Спрощення тестування.

CyberBionic systematics
CyberBionic Systematics ®
19 Eugene Sverstyuk Str., 5 floor
Kyiv, Ukraine
t. 0 800 337 146
E-mail: edu@cbsystematics.com
Site: www.edu.cbsystematics.com | itvdn.com

Закріплення матеріалу

- Що таке абстрактний клас?
- Що таке інтерфейс?
- Чим абстрактний клас відрізняється від інтерфейсу?
- Що таке множинне успадкування?
- Чим абстрактний клас відрізняється від конкретного?
- Які члени можуть бути абстрактними?

Додаткове завдання

Напишіть клас ConverterTemperature для конвертації з градусів за Цельсієм в Кельвіни, Фаренгейти. Клас має мати метод convert, який і робить конвертацію.

Самостійна діяльність учня

Завдання 1
Вивчіть основні конструкції та поняття, розглянуті на уроці.

Завдання 2
Використовуючи IntelliJ IDEA, створіть проєкт. Потрібно:
Створіть клас AbstractHandler.
У тілі класу створити методи void open(), void create(), void change(), void save().
Створити похідні класи XMLHandler, TXTHandler, DOCHandler від базового класу AbstractHandler.
Написати програму, яка виконуватиме визначення документа. Для кожного формату мають бути методи відкриття, створення, редагування, збереження певного формату документа.

Завдання 3
Використовуючи IntelliJ IDEA, створіть проєкт. Потрібно:
Створіть 2 інтерфейси Playable та Recodable. У кожному з інтерфейсів створіть по 3 методи void play() / void pause() / void stop() та void record() / void pause() / void stop() відповідно.
Створіть похідний клас Player від базових інтерфейсів Playable та Recodable.
Написати програму, яка виконує програвання та запис.

Завдання 4
Зайдіть на сайт Oracle.
Використовуючи пошукові механізми Oracle, знайдіть самостійно опис теми за кожним прикладом, який був розглянутий на уроці, так, як це наведено нижче, у розділі «Рекомендовані ресурси», описи цього уроку. Збережіть посилання та дайте їм короткий опис.

Рекомендовані ресурси

Абстрактні класи abstract
https://docs.oracle.com/javase/tutorial/java/IandI/abstract.html

Інтерфейси
http://msdn.microsoft.com/ru-ru/library/ms173156.aspx

CyberBionic systematics
CyberBionic Systematics ®
19 Eugene Sverstyuk Str., 5 floor
Kyiv, Ukraine
t. 0 800 337 146
E-mail: edu@cbsystematics.com
Site: www.edu.cbsystematics.com | itvdn.com